



Centro Integrado de Formación
Profesional nº 1

UT 1 Programación de Procesos

Curso: DAMSV_2A 2026-2027

Profesor: Miguel Ángel García Cejudo (miguelangel.garcia@educeuta.es)

Índice

Objetivos	4
1.1 Programa y procesos	4
1.2 Multitarea	5
1.3. Procesos y sistemas monoprocesadores y multiprocesadores	8
1.3.1. Sistemas monoprocesadores	8
1.3.2. Sistemas multiprocesadores	8
A. Sistemas fuertemente acoplados	8
B. Sistemas débilmente acoplados	9
1.4. Ventajas e inconvenientes de la programación concurrente	10
1.5. Kernel (o núcleo) del sistema operativo y llamadas al sistema	11
1.6. Estados de ejecución de un proceso	11
1.7. Hilos y procesos	13
1.8. Servicios	13
1.8.1. Servicios en Windows	14
1.8.2. Servicios en Linux	15
1.9. Concurrencia en Java	17
1.9.1. La clase Runtime	17
1.9.2. Las clases Process y ProcessBuilder	18
Actividades propuestas	21
1.9.3. Redirección de entrada y salida estándar y de error	23
A) Mecanismos de redirección de la clase ProcessBuilder	24
Actividades propuestas	26
B) Mecanismos de redirección a streams de la clase Process	26
RESUMEN	28

Ejercicios propuestos 29

Objetivos

- Conocer los conceptos de programa, proceso e hilo, y la relación entre ellos.
- Comprender cómo la técnica del cambio de contexto hace posible la multitarea o ejecución simultánea de varios procesos en un mismo procesador, y un mejor aprovechamiento de este.
- Entender los estados de ejecución de un proceso y su planificación a corto, medio y largo plazo.
- Identificar los distintos tipos de sistemas multiprocesadores, y la forma en la que los diferentes procesadores pueden intercambiar información entre sí.
- Analizar la programación concurrente, la multiprogramación y la programación paralela.
- Resolver problemas concretos utilizando las clases de Java que permiten crear dinámicamente, y ejecutar, de manera concurrente, nuevos procesos.
- Utilizar, de manera efectiva, los mecanismos que proporciona Java para gestionar y redireccionar la entrada y la salida estándar y de error de nuevos procesos creados

1.1 Programa y procesos

Un programa contiene un conjunto de instrucciones que se pueden ejecutar directamente en una máquina. Esta puede ser una máquina física o, en el caso del lenguaje Java, una máquina virtual. Un programa es un objeto estático, normalmente almacenado en un fichero binario en un medio de almacenamiento secundario, como, por ejemplo, un disco duro.

Un proceso corresponde a una instancia de un programa en ejecución. Un proceso es una entidad dinámica.

La ejecución de un programa comienza con la creación y ejecución de un proceso, y un proceso puede crear nuevos procesos sobre la marcha.

Para ejecutar un programa, primero debe cargarse en la memoria. Durante su ejecución, un programa utiliza diversos recursos del sistema:

- Memoria principal. Consta de una secuencia de celdas de memoria, todas con la misma longitud en bits, y cada una identificada por su posición. Antes de comenzar su ejecución, un programa debe cargarse en un bloque de la memoria principal, que se asigna al proceso que se crea para ejecutar el programa. Este proceso también puede obtener más memoria, dinámicamente, durante su ejecución.

1.1 Programa y procesos

- Procesador o CPU (Central Processing Unit). Ejecuta el proceso que se crea una vez cargado el programa en la memoria. El procesador guarda, en un registro especial, el contador de programa (PC o program counter), la dirección en la memoria de la instrucción que se está ejecutando. Un proceso no puede operar directamente con los contenidos de la memoria. Estos deben antes traerse de ella y cargarse en registros del procesador. Las operaciones se realizan en una unidad aritmético-lógica o UAL (en inglés, arithmetic and logical unit o ALU). Su resultado se obtiene en un registro desde el que se puede transferir a una posición de la memoria.
- Dispositivos de entrada/salida (E/S). Los procesos comparten los dispositivos de E/S. Debe guardarse información acerca de a qué procesos se les ha otorgado acceso a un dispositivo de E/S, y del estado de las operaciones realizadas sobre él.

Toda la información asociada a un proceso se guarda en un bloque de control de proceso (en inglés, PCB o process control block). Si el mismo programa se ejecuta varias veces, se creará cada vez un nuevo proceso, y cada uno tendrá su propio PCB. os de programa, proceso e hilo, y la relación entre ellos.

1.1 Programa y procesos

Un proceso puede crear nuevos procesos durante su ejecución, que pueden, a su vez, crear nuevos procesos. De esta forma, a partir de un proceso inicial puede crearse una jerarquía de procesos.

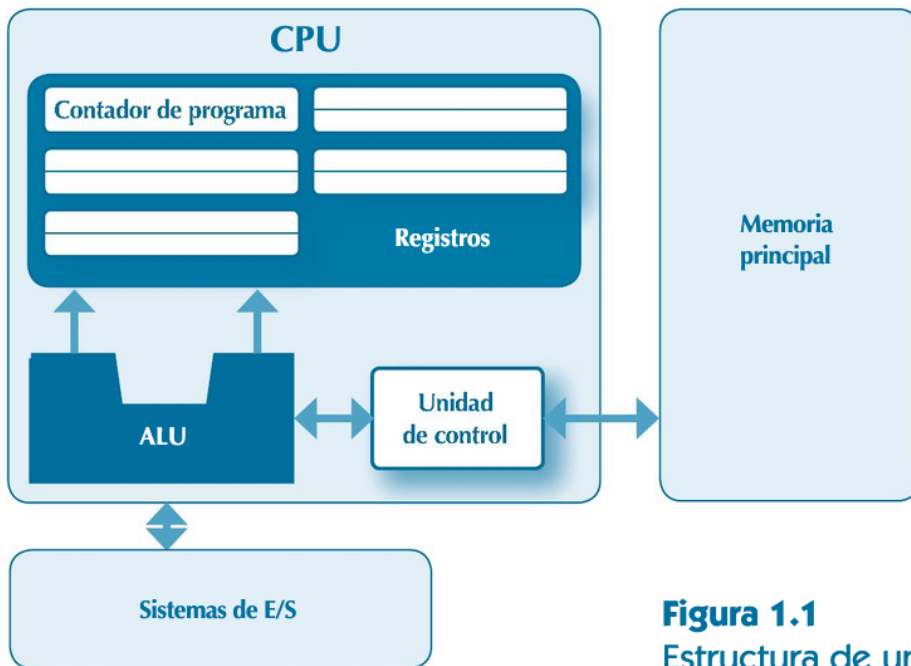


Figura 1.1
Estructura de un procesador.

Imagen de la diapositiva 5 de la presentación original.

1.2 Multitarea

Los sistemas operativos actuales permiten ejecutar múltiples procesos a la vez. Un usuario puede editar un fichero de texto, reproducir un vídeo, copiar ficheros de un directorio a otro, y descargar un fichero de internet. Todo ello, aunque exista un solo procesador en el sistema. A intervalos regulares de tiempo, la ejecución del proceso en curso se detiene y toma el control un programa especial del sistema operativo, que puede parar la ejecución del proceso actual y pasar a ejecutar otro proceso.

En cada momento hay un único proceso en ejecución, pero la rápida alternancia entre ellos hace que todos puedan seguir avanzando a lo largo del tiempo, de manera que considerando un intervalo de tiempo, se puede considerar que múltiples procesos se ejecutan

simultáneamente. Y así lo percibe un usuario del sistema.

CPU	A	A	A	A	A	B	B	B	B	C	C	C	C	C	C
Tiempo →	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Figura 1.2
Ejecución de procesos sin multitarea.

A															
B															
C															
CPU	A	B	C	A	B	C	A	B	C	A	B	C	A	C	C
Tiempo →	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Figura 1.3
Ejecución de procesos con multitarea.

Imagen de la diapositiva 7 de la presentación original.

1.2 Multitarea

.

1.2 Multitarea

Antes de empezar a ejecutar un nuevo proceso, se guarda el estado de ejecución del proceso actual, para poder más adelante retomar su ejecución en el mismo punto en que se interrumpió. En particular, se guardan los contenidos del contador de programa y de los registros del procesador, y se cargan los del nuevo proceso. Esto se conoce como cambio de contexto. El rendimiento de un sistema aumenta si dispone de más de un procesador (sistema multiprocesador). En este caso, se pueden ejecutar en paralelo varios procesos, uno en cada procesador..

1.2 Multitarea

CPU 1	A	A	A	A	A										
CPU 2	B	B	B	B											
CPU 3	C	C	C	C	C	C									
Tiempo →	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Figura 1.4
Ejecución de procesos en un sistema multiprocesador.

Imagen de la diapositiva 9 de la presentación original.

1.2 Multitarea

Pero, normalmente, siempre habrá muchos más procesos que procesadores, por lo que, también en el caso de los sistemas multiprocesadores, se hará multitarea en cada procesador. Se puede pensar

que, cuando solo se dispone de un procesador, no compensa la multitarea, porque, al tiempo de ejecución de los procesos, hay que sumar el tiempo necesario para los cambios de contexto, además del tiempo que necesita el sistema operativo para planificar la ejecución de los distintos procesos. Pero esto no es así, por los siguientes motivos:

- Tiempo dedicado: el tiempo que se dedica a esto es pequeño en comparación con el que se dedica a la ejecución de los procesos, y se puede justificar con las ventajas de ejecutar varios procesos al mismo tiempo, especialmente para sistemas interactivos en los que hay una frecuente interacción entre el usuario y el ordenador. La manera natural de trabajar de una persona es hacer varias cosas a la vez, y querrá que todas las tareas con las que está trabajando avancen, y una respuesta rápida cuando interactúa con cualquiera de ellas.

1.2 Multitarea

- Uso del procesador: durante la ejecución de los procesos, suele haber períodos muy largos de tiempo en que no hacen uso del procesador, por estar esperando a que se terminen de realizar operaciones de E/S. Un procesador de textos, por ejemplo, pasa la mayor parte del tiempo desocupado, a la espera de que el usuario pulse una tecla o realice una acción con el ratón. También pasa intervalos de tiempo desocupado mientras obtiene el contenido de un documento almacenado en un fichero, o graba un documento en un fichero.

La figura 1.5 muestra la ejecución de dos procesos que alternan periodos de uso del procesador con periodos de espera para que se realicen operaciones de E/S. Se puede ver que el tiempo total de ejecución es solo ligeramente superior al de cualquiera de ellos por separado, porque, en general, es posible utilizar el procesador para el otro proceso mientras uno está a la espera de que se complete una operación de E/S.

Proceso A

E/S																				
CPU																				
Tiempo →	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20

Proceso B

E/S																				
CPU																				
Tiempo →	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20

Multitarea con procesos A y B en un procesador

E/S		A	A	A		A	A			A	A	A	A	A			A	A			
CPU	A	B		B	A		B	A	A		B		B	B	A	A	B	B	A	A	B
Tiempo →	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21

Figura 1.5
Ejecución de procesos con multitarea, con operaciones de E/S.

Imagen de la diapositiva 12 de la presentación original.

1.2 Multitarea

1.3. Procesos y sistemas monoprocesadores y multiprocesadores

Un sistema monoprocesador tiene un único procesador, y un sistema multiprocesador, varios. En ambos tipos de sistemas, se pueden ejecutar varios procesos de manera simultánea. En este apartado se explican las técnicas utilizadas para ejecutar varios procesos concurrentes en ambos tipos de sistemas.

Si un programa lanza distintos procesos concurrentes para realizar una tarea, estos deben comunicarse y sincronizarse entre sí.

Las técnicas de comunicación y sincronización serán distintas, normalmente, según se trate de sistemas monoprocesadores o multiprocesadores, y según el tipo de sistema multiprocesador.

1.3. Procesos y sistemas monoprocesadores y multiprocesadores

1.3.1. Sistemas monoprocesadores

La ejecución concurrente de varios procesos en un sistema monoprocesador se conoce con el nombre de multiprogramación. Ya se ha visto que la técnica del cambio de contexto permite una multitarea efectiva y un mejor aprovechamiento del procesador, que se asigna a otro proceso durante el tiempo en que un proceso está inactivo, o a la espera de que se termine de realizar una operación de E/S.

1.3.2. Sistemas multiprocesadores

Los sistemas multiprocesadores disponen de más de un procesador. Se pueden clasificar, dependiendo de su arquitectura, en sistemas multiprocesadores fuertemente acoplados y sistemas multiprocesadores débilmente acoplados.

1.3. Procesos y sistemas monoprocesadores y multiprocesadores

A. Sistemas fuertemente acoplados

En estos sistemas existe una memoria compartida para todos los procesadores, a la que todos ellos acceden a través de un bus de conexión. También acceden al sistema de E/S a través del mismo bus.

Se pueden distinguir dos tipos de sistemas multiprocesadores fuertemente acoplados:

- **Sistemas multiprocesadores simétricos (SMP):** en los que los procesadores del sistema funcionan todos de igual manera, y no hay ninguno diferenciado del resto.
- **Sistemas multiprocesadores asimétricos:** en los que uno de los procesadores del sistema, el maestro o master, controla al resto de los procesadores (slave). Este tipo de sistemas también se denominan master-slave.

1.3. Procesos y sistemas monoprocesadores y multiprocesadores

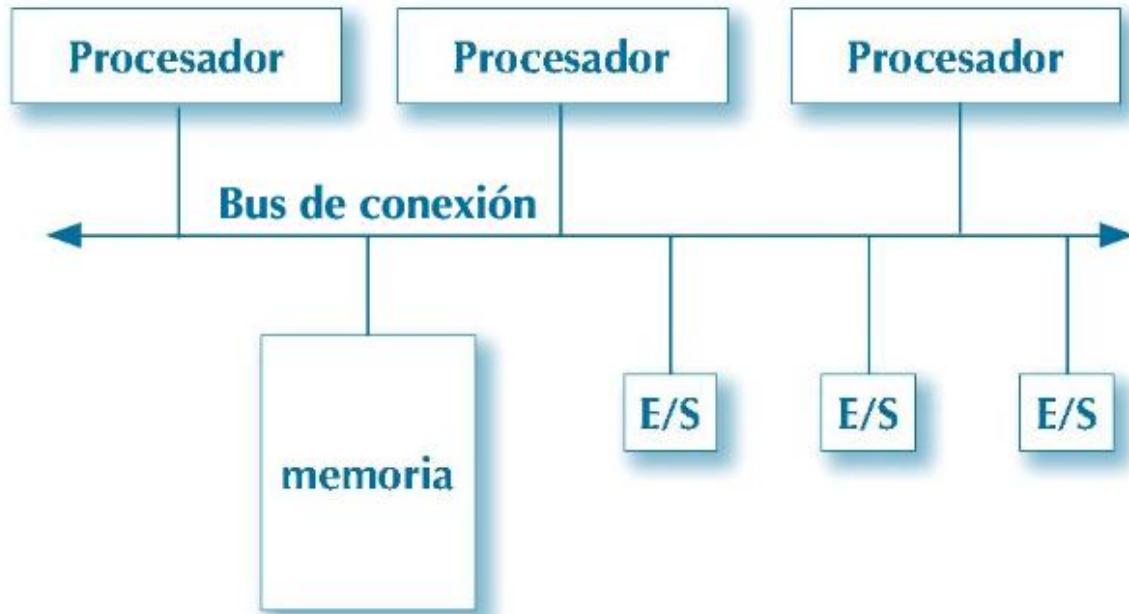


Imagen de la diapositiva 16 de la presentación original.

1.3. Procesos y sistemas monoprocesadores y multiprocesadores

B. Sistemas débilmente acoplados

En estos sistemas no hay memoria compartida, sino que cada procesador tiene su propia memoria y su propio sistema de E/S. Existe una red de comunicaciones entre los procesadores. Una posibilidad es que los procesadores estén físicamente cercanos unos a otros y comunicados mediante una red a medida, de alta velocidad y con una estructura regular. Este tipo de sistemas se utilizan en supercomputación y para aplicaciones especializadas intensivas en cálculo.

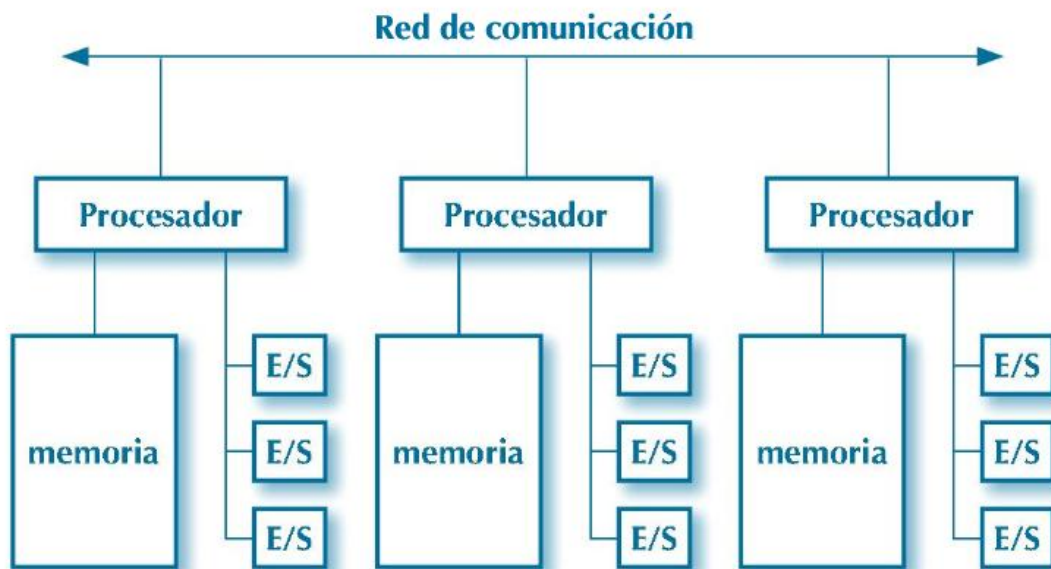


Imagen de la diapositiva 17 de la presentación original.

1.3. Procesos y sistemas monoprocesadores y multiprocesadores

Una alternativa a este tipo de sistemas son los clusters. Tienen también una estructura regular, pero, en lugar de procesadores de alto rendimiento interconectados mediante una red especializada y diseñada a medida, están formados por ordenadores convencionales conectados mediante una red de área local convencional.

Otra posibilidad, muy común hoy en día, es que los procesadores estén en ordenadores independientes comunicados mediante una red de área local (LAN) o incluso de área amplia (WAN) o a través de internet. Estos son los sistemas distribuidos. Se caracterizan por la heterogeneidad del hardware sobre el que están contruidos y por la relativa independencia entre los sistemas que los constituyen.

Las principales ventajas de los sistemas distribuidos es que son altamente escalables y configurables. Sus principales inconvenientes es que son sistemas heterogéneos y complejos de mantener. La sincronización entre procesos puede ser complicada también. La comunicación entre procesos se realiza a través de la red, que tiene una velocidad limitada, por lo que puede convertirse en un cuello de botella.

1.3. Procesos y sistemas monoprocesadores y multiprocesadores

Una alternativa a este tipo de sistemas son los clusters. Tienen también una estructura regular, pero, en lugar de procesadores de alto rendimiento interconectados mediante una red especializada y diseñada a medida, están formados por ordenadores convencionales conectados mediante una red de área local convencional.

Otra posibilidad, muy común hoy en día, es que los procesadores estén en ordenadores independientes comunicados mediante una red de área local (LAN) o incluso de área amplia (WAN) o a través de internet. Estos son los sistemas distribuidos. Se caracterizan por la heterogeneidad del hardware sobre el que están contruidos y por la relativa independencia entre los sistemas que los constituyen.

Las principales ventajas de los sistemas distribuidos es que son altamente escalables y configurables. Sus principales inconvenientes es que son sistemas heterogéneos y complejos de mantener. La sincronización entre procesos puede ser complicada también. La comunicación entre procesos se realiza a través de la red, que tiene una velocidad limitada, por lo que puede convertirse en un cuello de botella.

1.4. Ventajas e inconvenientes de la programación concurrente

La programación concurrente permite incrementar el rendimiento del sistema, porque los procesadores pasan a ejecutar otros procesos cuando el proceso que ejecutan está desocupado, o a la espera de que termine una operación de E/S. También hace posible un menor tiempo de respuesta, porque el tiempo de procesador se distribuye entre todos los procesos en ejecución. La mejora en el rendimiento dependerá, de todas formas, del tipo de aplicaciones

Lo ideal es que una aplicación se pueda descomponer en procesos independientes que se ejecuten en paralelo con un mínimo de interacción y comunicación entre ellos.

Un sistema multiprocesador debe ser flexible para permitir distribuir dinámicamente la carga de trabajo entre los procesadores disponibles, y escalable para permitir que la potencia de cálculo aumente con la instalación de nuevos procesadores.

Un sistema multiprocesador redundante dispone de procesadores suplementarios que proporcionan alta disponibilidad (bajo tiempo de respuesta ante peticiones de los usuarios) y tolerancia a fallos, de forma que si uno falla se deshabilita para continuar funcionando con el resto.

Los principales inconvenientes de la programación concurrente surgen de la dificultad para implementar mecanismos adecuados de sincronización y comunicación, pudiendo la sobrecarga anular la mejora en el rendimiento. En los sistemas distribuidos, la red puede convertirse en un cuello de botella si no posee la capacidad suficiente para el tráfico generado por el intercambio de mensajes entre procesos

1.5. Kernel (o núcleo) del sistema operativo y llamadas al sistema

La parte central del sistema operativo se denomina kernel o núcleo. Es una parte pequeña y muy optimizada del sistema operativo que da respuesta a multitud de eventos mediante un mecanismo de gestión de interrupciones.

Las interrupciones pueden producirse por diversas causas:

- • Eventos generados por el hardware. Por ejemplo: movimientos del ratón o pulsaciones del teclado, o el final de una operación de lectura o escritura en disco.
- • Llamadas al sistema. El kernel proporciona acceso para las aplicaciones a determinados servicios o funcionalidades del sistema operativo mediante llamadas al sistema.
- • Interrupciones periódicas que invocan al planificador de procesos a corto plazo del sistema operativo (CPU scheduler). Este decide si se sigue ejecutando el proceso actual o se hace un cambio de contexto para pasar a ejecutar otro proceso.

Cuando sucede una interrupción, el procesador deja de ejecutar el proceso en curso y pasa a ejecutar una rutina de tratamiento de interrupción. Cuando esta finaliza, se reanuda la ejecución en el mismo lugar en que estaba antes de la interrupción.

El procesador tiene dos modos de funcionamiento: modo kernel, o supervisor, y modo usuario. Las rutinas de tratamiento de interrupciones se ejecutan en modo kernel.

1.6. Estados de ejecución de un proceso

Desde que se crea inicialmente un proceso, puede pasar por diversos estados y cambiar de uno a otro. El sistema operativo gestiona los procesos y realiza los cambios de estado, teniendo en cuenta distintos eventos que pueden suceder durante el ciclo de vida del proceso.

Los cambios de estado de los procesos están controlados por el sistema operativo, con el objetivo de conseguir, por un lado, un máximo aprovechamiento de los recursos del sistema, y en particular del procesador o procesadores; y, por otro lado, una ejecución lo más eficiente posible de todos los procesos. Esto se consigue con una planificación a varios niveles:

- • Planificador a largo plazo o de trabajos (job scheduler). Un proceso debe estar en la memoria principal para poderse ejecutar. El planificador a largo plazo decide qué procesos son admitidos para su ejecución. Estos se cargan en la memoria principal y pasan a estado listo (ready). En sistemas operativos interactivos, como los que manejan la mayoría de los usuarios en sistemas de sobremesa, no hay propiamente planificación a largo plazo, sino que son los propios usuarios

quienes lanzan las aplicaciones, y estas se cargan inmediatamente en la memoria, y pasan a estado listo. Es importante, en cambio, en sistemas donde hay muchos procesos no interactivos y es el propio sistema operativo el que gestiona su lanzamiento.

1.6. Estados de ejecución de un proceso

- Planificador a corto plazo o de procesador (CPU scheduler). Se realiza para procesos cargados en la memoria principal. Su objetivo es repartir el tiempo de procesador entre todos los procesos, de manera que se consiga un máximo aprovechamiento del procesador, y que los procesos se ejecuten de la manera más eficiente posible. Un proceso en estado listo puede pasar a estado en ejecución y empezar a ejecutarse en el procesador. Un proceso en este estado puede volver a estado listo para permitir que otro proceso se ejecute. Cuando un proceso en ejecución realiza una operación de E/S, pasa a estado bloqueado, y se pasa a ejecutar un nuevo proceso cuyo estado cambia de listo a en ejecución. Cuando la operación de E/S haya concluido, el proceso volverá a estado listo. El planificador a corto plazo funciona basándose en interrupciones periódicas que hacen que el procesador pase inmediatamente a ejecutar una rutina de gestión de interrupción, que es parte del núcleo o kernel del sistema operativo. Esta rutina realiza la planificación a corto plazo, y decide si hay que continuar con la ejecución del proceso actual, o si hay que pasar a ejecutar un nuevo proceso y, en su caso, cuál.

1.6. Estados de ejecución de un proceso

- Planificador a medio plazo. Gestiona el paso de procesos de la memoria principal a la secundaria (suspensión) y viceversa (reanudación). Los sistemas operativos actuales funcionan, en general, con memoria virtual. Esta es una técnica que utiliza un espacio de direcciones virtual, mayor que el disponible en la memoria física, y utiliza para ello un medio de almacenamiento secundario. El contenido de cualquier dirección de la memoria virtual puede estar en la memoria física (memoria principal) o en almacenamiento secundario (memoria secundaria). Pero el microprocesador solo puede acceder directamente a la memoria principal. Cuando un proceso necesita más memoria de la que hay disponible en ella, se consigue pasando parte de sus contenidos a la memoria secundaria. Cuando se necesitan contenidos que están en la memoria secundaria, deben pasarse antes a la memoria principal. Si no hay suficiente espacio en ella, antes hay que pasar algunos de sus contenidos a la memoria secundaria. Se produce así un intercambio o swap entre las memorias principal y secundaria. La memoria virtual funciona bien si los intercambios solo se producen ocasionalmente, porque las operaciones de lectura y escritura en almacenamiento secundario son muy lentas. Si son demasiado frecuentes, se produce un trasiego constante entre las memorias principal y secundaria (trashing), y el rendimiento del sistema se degrada enormemente. Los procesos en la memoria principal que no se están ejecutando pueden suspenderse, y pasan a la memoria secundaria en los estados suspendido listo y suspendido bloqueado. Desde el estado suspendido bloqueado puede pasar al estado suspendido listo si se completa la operación de E/S que provocó su paso a estado bloqueado. También pueden reanudarse los procesos en estos estados para pasar a la memoria principal.

1.6. Estados de ejecución de un proceso

Para la gestión de los procesos, el sistema operativo utiliza un bloque de control de procesos (PCB) para cada proceso. El sistema operativo mantiene colas de procesos para cada uno de los estados. También mantiene colas para cada dispositivo de E/S, con información de los procesos que tienen operaciones pendientes en dicho dispositivo. La utilización de estas colas permite realizar, de manera muy eficiente, la planificación de procesos.

1.6. Estados de ejecución de un proceso

Desde que se crea inicialmente un proceso, puede pasar por diversos estados y cambiar de uno a otro. El sistema operativo gestiona los procesos y realiza los cambios de estado, teniendo en cuenta distintos eventos que pueden suceder durante el ciclo de vida del proceso,

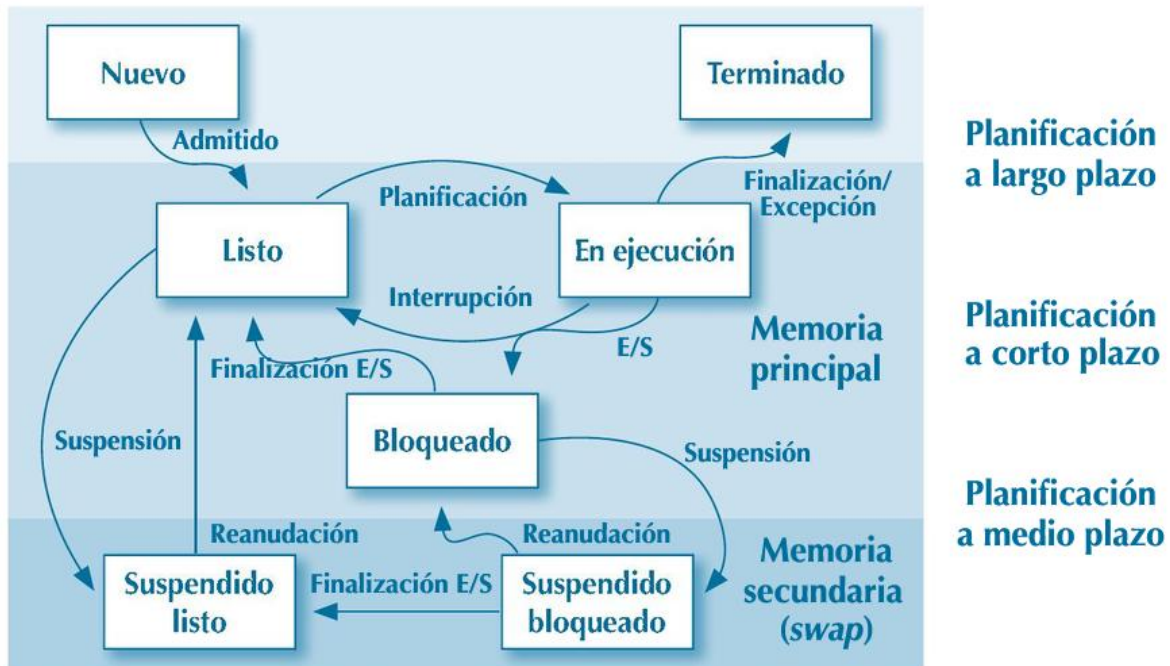


Imagen de la diapositiva 26 de la presentación original.

1.7. Hilos y procesos

Hasta ahora se ha venido hablando de procesos. Para crear un proceso es necesario reservar espacio en la memoria para él, lo que consume una cantidad considerable de tiempo y recursos del sistema, requiriendo mecanismos especiales de comunicación.

Los hilos o hebras (threads en inglés) se denominan a veces procesos ligeros (lightweight processes). Un proceso en ejecución tiene inicialmente un hilo, pero se pueden crear más de manera sencilla y rápida. La ejecución de un proceso termina cuando finalizan todos sus hilos, y a la inversa, si se termina un proceso, se acaba la ejecución de todos sus hilos.

La creación de un nuevo hilo para un proceso existente no requiere reservar ni inicializar espacio en la memoria, ya que este es compartido por todos los hilos del mismo proceso. Esto simplifica la comunicación: si un hilo modifica un objeto en la memoria, los demás hilos pueden ver los cambios. Por ello, se necesitan mecanismos de sincronización para evitar problemas al modificar objetos en la memoria compartida sin control.

El planificador a corto plazo o CPU scheduler gestiona de manera independiente los distintos hilos de un mismo proceso.

1.8. Servicios

Los servicios son un tipo particular de procesos que proporcionan determinados servicios (de ahí su nombre) a otros procesos. Se ejecutan en segundo plano (en background) y no son directamente utilizados por los usuarios.

Otros procesos, en cambio, se ejecutan en primer plano (en foreground), y tienen una interfaz de usuario con la que interactúan los usuarios.

Los servicios se ejecutan continuamente. Normalmente, son iniciados por el sistema operativo durante su arranque, y se ejecutan en segundo plano (en background). Suelen proporcionar información acerca de su ejecución en ficheros de log, en los que van anotando todos los eventos que se producen durante su ejecución y detalles acerca de las acciones que realizan.

Los servicios pueden proporcionar servicios a otros procesos en el mismo ordenador, y también a otros procesos en otros ordenadores. En este caso, suelen comunicarse a través de una red de

comunicaciones, utilizando protocolos de red estándar, como TCP o UDP, de la familia de protocolos TCP/IP. Este tipo de servicios en red son un componente fundamental de los sistemas distribuidos. En este contexto, las aplicaciones servidoras, o simplemente los servidores, prestan servicio a otras aplicaciones, y las aplicaciones clientes, o simplemente los clientes, utilizan estos servicios. Por supuesto, una aplicación puede proporcionar un servicio a otras aplicaciones y a la vez utilizar los servicios que proporciona otra aplicación. Es decir, puede actuar a la vez como cliente y como servidor.

1.8. Servicios

Una técnica frecuentemente utilizada por los servicios para mejorar el tiempo de respuesta ante peticiones de servicios de otros procesos es crear un nuevo hilo para responder a cada nueva petición recibida, o bien mantener un pool de hilos, de manera que a cada nueva petición se le puede asignar inmediatamente un hilo ya disponible en el pool. De esta forma, el hilo principal queda libre para recibir nuevas peticiones. Un servidor de este tipo es un servidor multihilo. Cada sistema operativo tiene sus propios mecanismos para gestionar los servicios. Normalmente, se puede configurar cada uno para que arranque automáticamente cuando arranca el sistema. También existen mecanismos generales para arrancar o parar servicios.

1.8.1. Servicios en Windows

En Windows, el administrador de servicios permite arrancar y detener servicios, así como consultar el estado de ejecución y obtener información adicional acerca de cada uno. También se puede configurar el modo de arranque para cada servicio: automático (para que arranque automáticamente cuando arranca el sistema operativo) o manual.

Administración de servicios de Windows

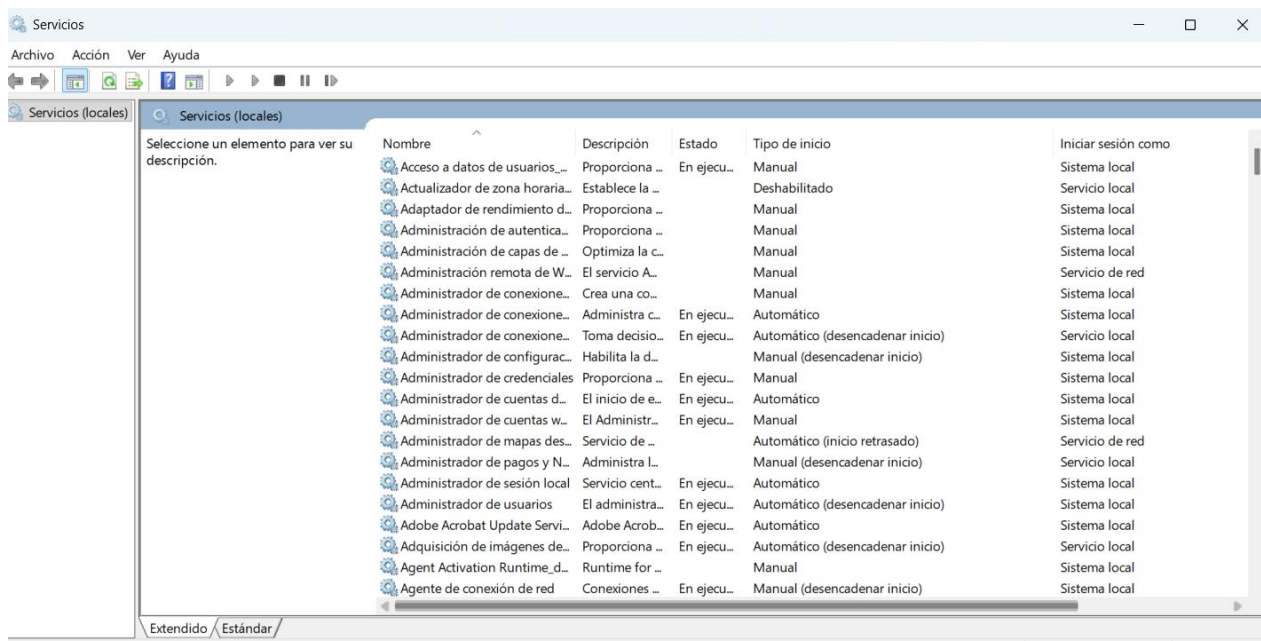


Imagen de la diapositiva 30 de la presentación original.

1.8. Servicios

1.8. Servicios

1.8.2. Servicios en Linux

En Linux existen sofisticados sistemas para gestión, control y configuración de servicios. El antiguo sistema initd de SystemV está siendo desplazado por el nuevo systemd. Para systemd se utiliza el comando systemctl.

Con el comando

```
$ systemctl list-unit-files,
```

se muestra una lista con todos los servicios existentes en el sistema.

Aquellos para los que se muestra enabled en la columna STATE se arrancan automáticamente cuando arranca el sistema.

Con el comando

```
$ systemctl --type=service,
```

se muestran todos los servicios cargados en el sistema. Que estén cargados no significa que estén en ejecución actualmente; el estado de ejecución aparece en la columna SUB.

..

```

root@LAPTOP-C4FP96C5: ~
root@LAPTOP-C4FP96C5:~# systemctl list-unit-files
UNIT FILE                                STATE      PRESET
proc-sys-fs-binfmt_misc.automount        static      -
dev-hugepages.mount                      static      -
dev-mqueue.mount                         static      -
proc-sys-fs-binfmt_misc.mount            disabled    disabled
snap.mount                               generated   -
sys-fs-fuse-connections.mount            static      -
sys-kernel-config.mount                  static      -
sys-kernel-debug.mount                   static      -
sys-kernel-tracing.mount                 static      -
apport-autoreport.path                   enabled     enabled
systemd-ask-password-console.path        static      -
systemd-ask-password-wall.path           static      -
session-1.scope                          transient   -
apparmor.service                        enabled     enabled
apport-autoreport.service                 static      -
apport-coredump-hook@.service            static      -
apport-forward@.service                  static      -
apport.service                          enabled     enabled
apt-daily-upgrade.service                 static      -
apt-daily.service                        static      -
apt-news.service                         static      -
autovt@.service                          alias       -
cloud-config.service                    enabled     enabled
cloud-final.service                     enabled     enabled
cloud-init-hotplugd.service              static      -
cloud-init-local.service                 enabled     enabled
cloud-init.service                      enabled     enabled
console-getty.service                   masked-runtime disabled

```

Imagen de la diapositiva 32 de la presentación original.

1.8. Servicios

1.8.2. Servicios en Linux

1.8. Servicios

```

root@LAPTOP-C4FP96C5:~# systemctl --type=service
UNIT                                LOAD    ACTIVE SUB    DESCRIPTION
console-setup.service              loaded active exited Set console font and keymap
cron.service                       loaded active running Regular background program processing daemon
dbus.service                       loaded active running D-Bus System Message Bus
getty@tty1.service                 loaded active running Getty on tty1
keyboard-setup.service             loaded active exited Set the console keyboard layout
kmod-static-nodes.service          loaded active exited Create List of Static Device Nodes
rsyslog.service                   loaded active running System Logging Service
setvtrgb.service                  loaded active exited Set console scheme
snapd.seeded.service              loaded active exited Wait until snapd is fully seeded
systemd-journal-flush.service      loaded active exited Flush Journal to Persistent Storage
systemd-journald.service           loaded active running Journal Service
systemd-logind.service             loaded active running User Login Management
systemd-modules-load.service       loaded active exited Load Kernel Modules
systemd-remount-fs.service         loaded active exited Remount Root and Kernel File Systems
systemd-resolved.service           loaded active running Network Name Resolution
systemd-sysctl.service             loaded active exited Apply Kernel Variables
systemd-timesyncd.service          loaded active running Network Time Synchronization
systemd-tmpfiles-setup-dev-early.service loaded active exited Create Static Device Nodes in /dev gracefully
systemd-tmpfiles-setup-dev.service loaded active exited Create Static Device Nodes in /dev
systemd-tmpfiles-setup.service     loaded active exited Create Volatile Files and Directories
systemd-udev-trigger.service       loaded active exited Coldplug All udev Devices
systemd-udevd.service              loaded active running Rule-based Manager for Device Events and Files
systemd-update-utmp.service        loaded active exited Record System Boot/Shutdown in UTMP
systemd-user-sessions.service      loaded active exited Permit User Sessions

```

Imagen de la diapositiva 33 de la presentación original.

1.8.2. Servicios en Linux

1.8. Servicios

1.8.2. Servicios en Linux

Para cada servicio, se pueden ejecutar comandos del tipo

```
$ systemctl acción servicio,
```

donde se indica la acción a realizar sobre el nombre del servicio.

Acciones principales disponibles:

- **status:** Muestra el estado de ejecución del servicio.
- **start:** Arranca el servicio si no se encuentra en ejecución.
- **stop:** Detiene el servicio si está en marcha.
- **restart:** Reinicia el servicio (lo detiene y lo vuelve a arrancar).
- **reload:** Actualiza la configuración del servicio sin necesidad de reiniciarlo por completo.
- **enable:** Configura el servicio para que se inicie de forma automática con el sistema.
- **disable:** Desactiva el arranque automático del servicio.

1.9. Concurrencia en Java

La máquina virtual de Java, el lenguaje Java y la biblioteca de clases estándar de Java se han diseñado para soportar programación concurrente.

La funcionalidad básica para procesos la proporciona la clase `Process`.

La funcionalidad básica para hilos la proporciona la clase `Thread`. Esta clase implementa la interfaz `Runnable`.

El paquete `java.util.concurrent` proporciona funcionalidad de alto nivel para programación concurrente. Incluye varios frameworks y algunas clases adicionales.

El resto de este capítulo se dedica a los procesos. En la siguiente unidad se dedica a los hilos (threads). La clase `Process` es abstracta y, por lo tanto, no se pueden crear objetos de ella, sino de subclases suyas, que, de hecho, implementan procesos nativos, es decir, procesos de plataformas específicas que se ejecutan en un sistema operativo particular para un tipo de procesador particular. La clase `Process` tiene métodos para lanzar un proceso, obtener información acerca de su estado y controlar su ejecución. Se pueden crear objetos de la clase `Process` con métodos de otras clases, en particular con

`ProcessBuilder.start()` y `Runtime.exec()`.

1.9. Concurrencia en Java

1.9.1. La clase `Runtime`

Cada programa de Java tiene asociada una instancia de la clase `Runtime` que le permite obtener información de su entorno de ejecución e interactuar con él. Este se puede recuperar con el método estático `getRuntime()`. Las diversas variantes del método `exec()` permiten lanzar nuevos procesos. En el siguiente cuadro se muestran algunos de los métodos de esta clase.

Método	Funcionalidad
<code>static Runtime getRuntime()</code>	Devuelve el objeto <code>Runtime</code> asociado con la aplicación de Java actual.
<code>Process exec(String command)</code>	Ejecuta un comando. Algunas variantes permiten especificar el
<code>Process exec(String[] comandoYArgs)</code>	entorno de ejecución y el directorio para ejecutar el comando.
<code>Process exec(String[] comandoYArgs, String[] envp)</code>	
<code>Process exec(String[] cmdarray, String[] envp, File dir)</code>	

Método	Funcionalidad
<code>void exit(int status)</code>	Termina la ejecución de la máquina virtual. El primer método lo
<code>void halt(int status)</code>	hace de manera ordenada, y el segundo, de manera abrupta.
<code>int availableProcessors()</code>	Devuelve el número de procesadores disponibles para la máquina virtual de Java.
<code>long freeMemory()</code>	Devuelve la cantidad de memoria disponible para la máquina virtual.

1.9. Concurrencia en Java

1.9.2. Las clases `Process` y `ProcessBuilder`

`Process` es una clase abstracta. Se pueden obtener instancias de subclases tuyas que proporcionan implementaciones para una plataforma particular, con métodos de la clase `ProcessBuilder`. Con `ProcessBuilder` se puede configurar previamente el entorno de ejecución de los procesos que crea y, en particular, como se verá en breve, redirigir su entrada y salida.

En el cuadro se muestran algunos de sus métodos de la clase `ProcessBuilder`

Método	Funcionalidad
<code>ProcessBuilder(String... comando)</code>	Construye un <code>ProcessBuilder</code> para un comando con los
<code>ProcessBuilder(List<String> comando)</code>	correspondientes argumentos de línea de comandos, en su caso
<code>ProcessBuilder directory(File directorio)</code>	El primer método establece el directorio de trabajo del
<code>File directory()</code>	proceso, y el segundo método lo devuelve. Por defecto, el segundo método normalmente devuelve null. Si es así, se puede obtener el directorio de ejecución del proceso actual con <code>System.getProperty("user.dir")</code> .
<code>Map<String, String> environment()</code>	Devuelve el entorno de trabajo del proceso, que es una lista de asignación de valores para variables de entorno.
<code>Process start()</code>	Crea e inicia un proceso

1.9. Concurrencia en Java

1.9.2. Las clases `Process` y `ProcessBuilder`

El siguiente fragmento de código no funciona:

```
ProcessBuilder pb = new ProcessBuilder("df -h /");
pb.start();
```

Produce la siguiente excepción:

```
java.io.IOException: Cannot run program "df -h /": error=2, No existe el archivo o el directorio
```

Esto es porque `ProcessBuilder` tiene constructores para una lista de `String` o para varios `String`, e interpreta el primer `String` como el programa que ejecutar, y el resto, como parámetros de línea de

comandos. Si solo se le pasa "df -h /", intenta ejecutar un programa "df -h /", que no existe (sí existe, en cambio, el programa "df"). La ejecución del código anterior en Java equivale a lo siguiente en la línea de comandos de Linux:

```
$ "df -h /"
bash: df -h /: No existe el archivo o el directorio
```

Sí funciona, en cambio, si se usa cualquiera de los siguientes constructores:

```
ProcessBuilder pb = new ProcessBuilder("df", "-h", "/");
ProcessBuilder pb = new ProcessBuilder(new String[] {"df", "-h", "/"});
```

Si el comando que ejecutar viene dado en un String `s`, se puede obtener un array de String con el método `split()` de la clase String. Por ejemplo:

```
String comando = "df -h /";
ProcessBuilder pb = new ProcessBuilder(comando.split(" +"));
```

"+" es una expresión regular para una secuencia de espacios (uno o más espacios).

1.9. Concurrencia en Java

1.9.2. Las clases Process y ProcessBuilder

Los principales métodos de la clase Process.

Método	Funcionalidad
<code>void destroy()</code> <code>public Process destroyForcibly()</code>	Termina el proceso. El primer método permite una terminación limpia y ordenada del proceso. El segundo lo termina inmediatamente.
<code>int exitValue()</code>	Devuelve el valor de salida, o código de retorno, del proceso. Por convención, un valor 0 indica terminación normal, y otro valor se interpretará como un código de error. Se puede terminar un programa en Java con un código de retorno distinto de 0 con <code>System.exit(código)</code>
<code>ProcessHandle.Info info()</code>	Devuelve la información actual del proceso
<code>boolean isAlive()</code>	Comprueba si el proceso está vivo
<code>long pid()</code>	Devuelve el PID o identificador de proceso.
<code>int waitFor()</code>	Hace que el hilo en ejecución espere hasta que el proceso haya terminado. Devuelve el valor de salida del proceso. Un valor cero se entiende que corresponde a una ejecución sin errores, mientras que un valor distinto de cero corresponde a un código de error. Si el proceso es de un programa en Java, es el valor devuelto por <code>System.exit()</code> , o cero si no se terminó la ejecución con <code>System.exit()</code>
<code>boolean waitFor(long timeout, TimeUnit unit)</code>	Hace que el hilo en ejecución espere hasta que el proceso haya terminado, durante un tiempo máximo indicado por <code>timeout</code> . Devuelve true si el proceso ha terminado por sí mismo antes del tiempo máximo indicado, y false en caso contrario.

1.9. Concurrencia en Java

1.9.2. Las clases Process y ProcessBuilder

Cualquiera de los múltiples errores que se pueden producir durante la creación y ejecución del proceso hará que se produzca una excepción, en general, de la clase `IOException`.

El siguiente programa de ejemplo crea un proceso para un comando que se pasa como parámetros de línea de comandos en args. Estos se pasan directamente al creador de ProcessBuilder con ProcessBuilder(args). La llamada a inheritIO() hace que el proceso herede la entrada y salida estándar del proceso padre. Con waitFor() se espera a que termine la ejecución del proceso hijo y se obtiene el código de retorno. Si, mientras eso sucede, se interrumpe la ejecución del programa, se lanzaría una excepción de tipo InterruptedException, que este programa capturaría, y de la que informaría.

1.9. Concurrency en Java

1.9.2. Las clases Process y ProcessBuilder

```
import java.util.Arrays;
import java.io.IOException;
public class LanzaProceso {
    public static void main(String[] args) {
        if (args.length <= 0) {
            System.out.println("Debe indicarse comando a ejecutar.");
            System.exit(1);
        }
        ProcessBuilder pb = new ProcessBuilder(args);
        pb.inheritIO();
        try {
            Process p = pb.start();
            int codRet = p.waitFor();
            System.out.println("La ejecución de " + Arrays.toString(args)
                + " devuelve " + codRet
                + " " + (codRet == 0 ? "(ejecución correcta)" : "(ERROR)"));
        } catch (IOException e) {
            System.err.println("Error durante ejecución del proceso");
            System.err.println("Información detallada");
        }
    }
}
```

1.9. Concurrency en Java

1.9.2. Las clases Process y ProcessBuilder

```
try {
    Process p = pb.start();
    int codRet = p.waitFor();
    System.out.println("La ejecución de " + Arrays.toString(args)
        + " devuelve " + codRet
        + " " + (codRet == 0 ? "(ejecución correcta)" : "(ERROR)"));
} catch (IOException e) {
    System.err.println("Error durante ejecución del proceso");
    System.err.println("Información detallada");
    System.err.println("-----");
    e.printStackTrace();
    System.err.println("-----");
    System.exit(2);
} catch (InterruptedException e) {
    System.err.println("Proceso interrumpido");
    System.exit(3);
}
}
```

1.9. Concurrency en Java

1.9.2. Las clases Process y ProcessBuilder

Actividades propuestas

1.2. Prueba el programa de ejemplo anterior con los siguientes casos, y comenta su funcionamiento

- Ejecución de un proceso que se ejecuta sin errores. Por ejemplo, `ls /etc` en Linux
- Ejecución de programa que no existe (por ejemplo), `programa_que_no_existe`
- Ejecución de un proceso que termina con un código de error. Por ejemplo, `ls /et` en Linux

1.3. Crea un programa que lance un proceso y utilice el método `isAlive()` para comprobar si se sigue ejecutando. El programa debe comprobar cada 3 segundos si el proceso está en ejecución, hasta que ya no lo esté, y entonces debe terminar. Tras cada comprobación del estado de ejecución, debe mostrar un mensaje indicando ese estado

- Para hacer una pausa con una duración determinada, se puede utilizar `Thread.sleep(int tiempo_ms)`
- Quizá pueda servir de ayuda el siguiente *shell script*, que se puede ejecutar con `bash bucle.sh`, si

está en un fichero llamado `bucle.sh`. El programa puede lanzar este script, cuya ejecución dura unos 10 segundos

1.9. Concurrencia en Java

1.9.2. Las clases `Process` y `ProcessBuilder`

Actividades propuestas

1.2. Prueba el programa de ejemplo anterior con los siguientes casos, y comenta su funcionamiento

- Ejecución de un proceso que se ejecuta sin errores. Por ejemplo, `ls /etc` en Linux
- Ejecución de programa que no existe (por ejemplo), `programa_que_no_existe`
- Ejecución de un proceso que termina con un código de error. Por ejemplo, `ls /et` en Linux

1.3. Crea un programa que lance un proceso y utilice el método `isAlive()` para comprobar si se sigue ejecutando. El programa debe comprobar cada 3 segundos si el proceso está en ejecución, hasta que ya no lo esté, y entonces debe terminar. Tras cada comprobación del estado de ejecución, debe mostrar un mensaje indicando ese estado

- Para hacer una pausa con una duración determinada, se puede utilizar `Thread.sleep(int tiempo_ms)`
- Quizá pueda servir de ayuda el siguiente *shell script*, que se puede ejecutar con `bash bucle.sh`, si está en un

fichero llamado `bucle.sh`. El programa puede lanzar este script, cuya ejecución dura unos 10 segundos

```
i=0
while [ $i -lt 10 ]
do
echo $i
i=$((i + 1))
sleep 1
done
```

1.9. Concurrencia en Java

1.9.2. Las clases `Process` y `ProcessBuilder`

El siguiente programa de ejemplo establece un tiempo máximo para la ejecución de un comando que resulta insuficiente para su ejecución, por lo que se interrumpe. El proceso hijo hereda la entrada y salida del proceso padre con `inheritIO()`. Además, la salida de error se redirige hacia la salida estándar con `redirectErrorStream(true)`. Con esto se consigue que en la salida del programa ambas salgan secuencialmente. Si no se hace esto, el mensaje de que se ha terminado la ejecución del programa sale entremezclado con la salida de error, que muestra gran cantidad de mensajes de error, y pasa desapercibido. El array `{"find", "/", "-name", "\"*\""}"` representa el comando `find / -name "*"'`.

```
package ejecutacomandotimeout;
import java.util.Arrays;
import java.util.concurrent.TimeUnit;
import java.io.IOException;
public class EjecutaComandoTimeout {
    public static int MAX_TIEMPO = 500;
    public static void main(String[] args) {
```

```
    ProcessBuilder pb = new ProcessBuilder(
        new String[]{"find", "/", "-name", "\"*\""});
    System.out.printf("Se ejecuta comando: %s\n", Arrays.toString(args));
    pb.inheritIO();
```

1.9. Concurrencia en Java

1.9.2. Las clases *Process* y *ProcessBuilder*

```
    System.out.printf("Se ejecuta comando: %s\n", Arrays.toString(args));
    pb.inheritIO();
    pb.redirectErrorStream(true);
    try {
        Process p = pb.start();
        if (!p.waitFor(MAX_TIEMPO, TimeUnit.MILLISECONDS)) {
            p.destroy();
            System.out.printf("AVISO: No ha terminado en %d ms\n",
                MAX_TIEMPO);
        }
    } catch (IOException e) {
        System.err.println("Error durante ejecución. Información detallada");
        System.err.println("-----");
        e.printStackTrace();
        System.err.println("-----");
        System.exit(1);
    } catch (InterruptedException ex) {
        System.err.println("Proceso interrumpido");
        System.exit(2);
    }
}
```

1.9. Concurrencia en Java

1.9.2. Las clases *Process* y *ProcessBuilder*

Actividades propuestas

1.4. Crea un programa que muestre cuál es el directorio por defecto de ejecución de un proceso, con el método `directory()` de *ProcessBuilder*, y que ejecute el mismo comando (por ejemplo, `ls`) en distintos directorios, asignados con el método `directory(File directorio)` de *ProcessBuilder*.

1.5. Muestra el entorno de ejecución de un proceso con `Map<String, String> environment()` de

ProcessBuilder. Hay que iterar sobre el Map<String, String> devuelto para mostrar cada entrada.

1.9. Concurrency en Java

1.9.3. Redirección de entrada y salida estándar y de error

Las clases ProcessBuilder y Process proporcionan diversos mecanismos para redireccionar y gestionar la salida y entrada de los procesos. Esto podría no funcionar bien en algunas plataformas, debido a limitaciones en los buffers utilizados para ellas por el sistema operativo particular. Estos mecanismos se aprovechan en algunos de los programas de ejemplo que se presentan a continuación.

Los conceptos de entrada y salida estándar y de salida de error se introdujeron en el sistema operativo UNIX, y se implementaron posteriormente en Linux y Windows. Cualquier proceso tiene acceso a varios streams o flujos de bytes.

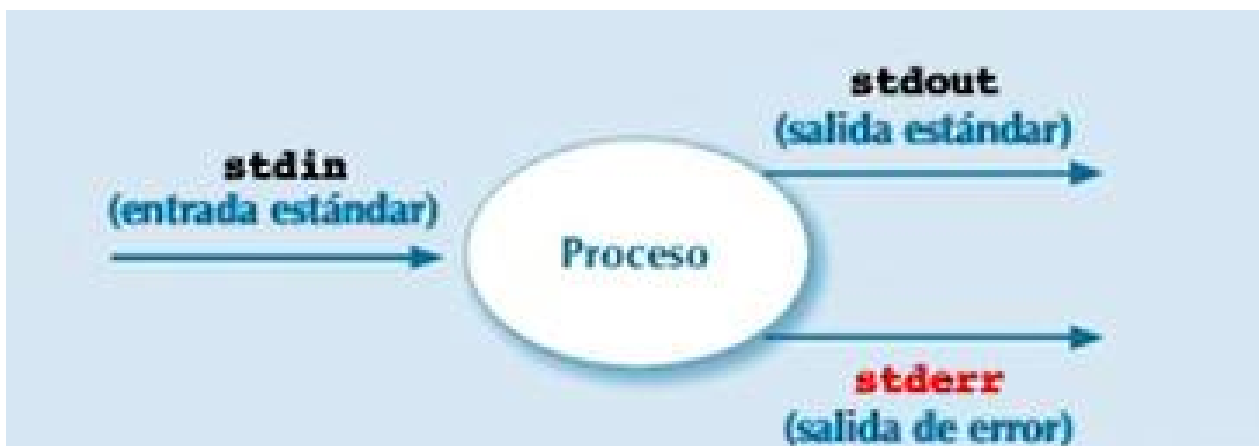


Imagen de la diapositiva 48 de la presentación original.

1.9. Concurrency en Java

1.9.3. Redirección de entrada y salida estándar y de error

- Entrada estándar o stdin (standard input): Es un stream de entrada del que puede leer datos. Normalmente, se leen datos sobre los que se realiza algún procesamiento. Por defecto, suele estar asociada al teclado, de manera que el programa puede leer todo el texto que se introduce por teclado.
- Salida estándar o stdout (standard output): Es un stream de salida en el que se pueden escribir datos. Una manera sencilla en que un programa puede generar su salida es escribirla en la salida estándar. Por defecto, suele estar asociada a la pantalla, de manera que todo el texto que se envía a la salida estándar se escribe en la pantalla.
- Salida de error o stderr (standard output): Es un stream de salida en el que se pueden escribir datos. También suele estar asociada a la pantalla, de manera que todo el texto que se envía a la salida de error se escribe en la pantalla.

El proceso puede ser cualquier programa en ejecución. Para los siguientes ejemplos se utilizarán comandos existentes en Linux.

1.9. Concurrency en Java

1.9.3. Redirección de entrada y salida estándar y de error

Existen mecanismos de redirección para estos streams muy potentes y versátiles, que se detallan a continuación, y para los que se ponen ejemplos.

	Fichero	Ejemplo
A entrada estándar desde:	proceso < fichero	grep localhost < /etc/hosts.
De salida estándar hacia:	proceso > fichero proceso >> fichero	(ls -l > listafich.txt), (echo -fin- >> listafich.txt).
De salida de error hacia:	proceso 2> fichero proceso 2>>	(ls /et 2> errores.log), fichero (ls /etcaetera 2>> errores.log).
(Nota: Para desechar salida, se puede redirigir a /dev/null).		
De salida estándar de un proceso a entrada estándar de otro:	proceso1 proceso2	head -4 /etc/fstab tail -1

1.9. Concurrencia en Java

1.9.3. Redirección de entrada y salida estándar y de error

A) Mecanismos de redirección de la clase ProcessBuilder

La clase ProcessBuilder proporciona mecanismos para la redirección de la entrada y salida de los nuevos procesos creados. En los programas de ejemplo vistos hasta ahora, se ha utilizado el método `inheritIO()` para enlazar la entrada y la salida estándares y de error de los subprocesos con las del proceso principal, y por eso se ha podido ver su salida.

Los mecanismos proporcionados por ProcessBuilder para ello son los siguientes:

- El método `inheritIO()`. Esta es la solución más sencilla y la utilizada en el programa anterior. Con este método, la entrada y la salida estándares y de error de los procesos creados se enlazan con las del proceso actual.
- Los métodos `redirectInput()`, `redirectOutput()` y `redirectError()` son útiles cuando se necesita una sencilla redirección, desde o hacia un fichero, o bien a `/dev/null` para ignorar la salida. `redirectInput()` redirige la entrada desde:
 - La entrada estándar del proceso padre con `redirect.INHERIT`.
 - El contenido de un fichero con un objeto `File`.

1.9. Concurrencia en Java

1.9.3. Redirección de entrada y salida estándar y de error

A) Mecanismos de redirección de la clase ProcessBuilder

- `redirectOutput()` redirige la salida estándar de los nuevos procesos creados hacia:
 - La salida estándar del proceso padre con `redirect.INHERIT`.
 - Un fichero, borrando contenidos actuales del fichero con `f`, de clase `File`.
 - Un fichero, añadiendo a sus contenidos actuales con `Redirect.appendTo(f)`.
 - Ninguna parte, con lo que se descarta sin más con `Redirect.DISCARD`.
- `redirectError()` funciona como `redirectOutput()`, pero para la salida de error.

El cuadro siguiente muestra la equivalencia entre los diversos métodos de redirección para la clase ProcessBuilder de Java y mecanismos de redirección disponibles para la ejecución de comandos en una shell de Linux.

1.9. Concurrency en Java

1.9.3. Redirección de entrada y salida estándar y de error

A) Mecanismos de redirección de la clase `ProcessBuilder`

- Métodos de redirección para clase `ProcessBuilder` de Java, se muestra todos los métodos para redirección de entrada y salida

Métodos	Línea de comandos de Linux
<code>redirectInput(new File(fichero))</code>	comando < fichero
<code>redirectOutput(new File(fichero))</code>	comando > fichero
<code>redirectOutput(Redirect.appendTo(new File(fichero)))</code>	comando >> fichero
<code>redirectOutput(Redirect.DISCARD)</code>	comando > /dev/null
<code>redirectError(new File(fichero))</code>	comando 2> fichero
<code>RedirectError(Redirect.appendTo(new File(fichero)))</code>	comando 2>> fichero
<code>redirectError(Redirect.DISCARD)</code>	comando 2> /dev/null

1.9. Concurrency en Java

1.9.3. Redirección de entrada y salida estándar y de error

A) Mecanismos de redirección de la clase `ProcessBuilder`

Métodos de la clase `ProcessBuilder` para redirección de entrada y salida

Método	Funcionalidad
<code>ProcessBuilder inheritIO()</code>	Redirige la salida estándar y de error de los subprocesos creados hacia las del proceso padre, y su entrada estándar desde la del proceso padre.
<code>ProcessBuilder redirectInput(File f)</code>	Redirige las entrada y salida estándares y de
<code>ProcessBuilder redirectInput(ProcessBuilder.Redirect fuente)</code>	error, respectivamente, desde o hacia: Un fichero (con <code>File f</code>). •
<code>ProcessBuilder redirectOutput(File f)</code>	La correspondiente del proceso padre
<code>redirectOutput(ProcessBuilder.Redirect destino)</code>	(con <code>Redirect.INHERIT</code>). La salida estándar y de error se pueden •
<code>ProcessBuilder redirectError(File f)</code>	descartar (con <code>Redirect.DISCARD</code>).
<code>redirectError(ProcessBuilder.Redirect destino)</code>	

1.9. Concurrency en Java

1.9.3. Redirección de entrada y salida estándar y de error

A) Mecanismos de redirección de la clase `ProcessBuilder`

Métodos de la clase `ProcessBuilder` para redirección de entrada y salida

Método	Funcionalidad
<code>ProcessBuilder redirectErrorStream(boolean redir):</code>	Con valor true para redir, dirige la salida de error hacia la estándar. Esto hace innecesario gestionar por separado la salida de error. Tiene el inconveniente de que no se puede distinguir entre ambas. Pero tiene la ventaja de que ambas aparezcan entremezcladas, lo que dificulta su lectura e interpretación
<code>static List<Process> startPipeline(List<ProcessBuilder> builders):</code>	Inicia un proceso para cada <code>ProcessBuilder</code> , y crea una secuencia de procesos cuyas salida y entradas están enlazadas mediante tuberías o pipes, de manera que la salida de un proceso se dirige a la entrada del siguiente en la secuencia. Solo se pueden redirigir la entrada del primer proceso y la salida del último. El resto de entradas y salidas están conectadas mediante pipes de un proceso al siguiente y, obviamente, no se pueden redirigir.

1.9. Concurrencia en Java

1.9.3. Redirección de entrada y salida estándar y de error

A) Mecanismos de redirección de la clase `ProcessBuilder`

Actividades propuestas

1.6. Crea un programa que ejecute el comando `grep` para seleccionar las líneas de un fichero que contengan un texto, y que las escriba en otro fichero. El texto, el fichero en el que se buscan las líneas que contienen el texto y el fichero en el que se escriben las líneas que contienen el texto se pasan como argumentos de línea de comandos. Puedes desarrollar el programa a partir del primer programa de ejemplo. Solo hay que utilizar adecuadamente los métodos `redirectInput` y `redirectError`.

1.7. Crea una secuencia de procesos que tenga como salida el resultado de ejecutar el comando `grep ^video /etc/mime.types | sed "s/\t\t*/:/g"`. Este comando muestra los tipos MIME para vídeo y sus tipos de fichero. Debe crearse una secuencia con ambos procesos (`grep ^video` y `sed "s/\t\t*/:/g"`), enlazados mediante una tubería, con `ProcessBuilder.startPipeline()`. Después, debe redirigirse a la entrada estándar del primero el contenido del fichero `/etc/fstab`, y redirigir la salida estándar del segundo hacia la propia.

1.9. Concurrencia en Java

1.9.3. Redirección de entrada y salida estándar y de error

B) Mecanismos de redirección a streams de la clase `Process`

La clase `Process` tiene métodos que devuelven streams asociados a la entrada estándar del proceso, a la salida estándar y a la salida de error. Esto permite enviar datos a la entrada estándar del proceso y leer de la salida estándar y de error, lo que permite un absoluto control y flexibilidad, más allá de los sencillos mecanismos de redirección vistos anteriormente.

1.9. Concurrencia en Java

1.9.3. Redirección de entrada y salida estándar y de error

B) Mecanismos de redirección a streams de la clase `Process`

Métodos de Process para obtener streams asociados a entrada estándar y a salidas estándar y de error

Método	Funcionalidad
InputStream getInputStream()	Devuelve un stream de entrada conectado con la salida estándar del proceso.
OutputStream getOutputStream()	Devuelve un stream de salida conectado con la entrada estándar del proceso.
InputStream getErrorStream()	Devuelve un stream de entrada conectado con la salida de error del proceso.

Sobre estos streams binarios se pueden construir streams de texto (un InputStreamReader sobre un InputStream), y sobre estos se pueden construir streams con buffering, que permiten leer o escribir líneas enteras.

El siguiente programa de ejemplo ejecuta un comando que se proporciona como argumento de línea de comandos. La salida del proceso p se obtiene con p.getInputStream(). Sobre el InputStream obtenido se construye un BufferedReader, para poder obtener la salida como texto

Programación de Servicios y Procesos 26-27 DAWSV-2A

1.9. Concurrency en Java

1.9.3. Redirección de entrada y salida estándar y de error

B) Mecanismos de redirección a streams de la clase Process

Actividad: Crea un programa al que se le pase como argumento de línea de comandos la ruta o path de un directorio. El path introducido debe ser el de un directorio, y si no existe, debe mostrarse un mensaje de error, y también si existe, pero corresponde a un fichero y no a un directorio. Para ello, debe utilizarse la clase File. Si existe, debe mostrarse el resultado del comando ls -l sobre ese directorio, pero las líneas deben ir numeradas, teniendo la primera el número 1. El programa debe obtener un stream asociado a la salida estándar del proceso y después leer línea a línea de él. Para ejecutar el proceso puede utilizarse la clase

ProcessBuilder, y también la clase Runtime, y solo cambia una línea del programa.

1.9. Concurrency en Java

1.9.3. Redirección de entrada y salida estándar y de error

B) Mecanismos de redirección a streams de la clase Process

El siguiente programa lee nombres de dominios de la entrada estándar. Cada uno lo envía hacia la entrada estándar del comando nslookup, que proporciona información acerca de él. Es importante cerrar, primero, el stream devuelto por p.getOutputStream() y, después, esperar a que termine el proceso con p.waitFor(). La salida se redirige con

pb.redirectOutput(ProcessBuilder.Redirect.INHERIT). Nada hubiera impedido obtenerla línea a línea de un stream obtenido con getInputStream(), como en el ejemplo anterior.

```

package lanzaprocesoEscribeEntrada;
import java.io.InputStreamReader;
import java.io.BufferedReader;
import java.io.OutputStream;
import java.io.OutputStreamWriter;
import java.io.IOException;
public class LanzaProcesoEscribeAEntrada {

```

1.9. Concurrency en Java

1.9.3. Redirección de entrada y salida estándar y de error

B) Mecanismos de redirección a streams de la clase Process

```

public class LanzaProcesoEscribeAEntrada {
    public static void main(String[] args) {
        ProcessBuilder pb = new ProcessBuilder("nslookup");
        pb.redirectOutput(ProcessBuilder.Redirect.INHERIT);
        try (InputStreamReader isstdin = new InputStreamReader(System.in, "UTF-8");
            BufferedReader brstdin = new BufferedReader(isstdin)) {
            String linea;
            System.out.print("Introducir nombre de dominio: ");
            while ((linea = brstdin.readLine()) != null && linea.length() != 0) {

```

```

p = pb.start(); // Lanza el proceso

```

1.9. Concurrency en Java

1.9.3. Redirección de entrada y salida estándar y de error

B) Mecanismos de redirección a streams de la clase Process

```

try (
        OutputStream osp = p.getOutputStream();
        OutputStreamWriter oswp = new OutputStreamWriter(osp, "UTF-8");
        oswp.write(linea); // Envía línea leída al proceso
    }
    try {
        p.waitFor();
    } catch (InterruptedException ex) { }
    System.out.print("Introducir nombre de dominio: ");
}
    } catch (IOException e) {
        System.out.println("ERROR: de E/S"); e.printStackTrace();
    }
}
}

```

RESUMEN

- Un programa es un objeto estático que contiene instrucciones y datos. Se suele almacenar en un fichero binario en un medio de almacenamiento secundario. Un proceso es un objeto dinámico que se genera cuando se carga un programa en la memoria principal y se ejecuta.
- La multitarea permite la ejecución de más de un proceso en un procesador. El procesador va ejecutando alternativamente distintos procesos. Para pasar de uno a otro, se realiza un cambio de contexto. La primera parte de este proceso consiste en salvaguardar el estado de ejecución actual del proceso que se está ejecutando actualmente, de manera que, después, se pueda restaurar para seguir con su ejecución. Esta técnica se utiliza tanto en sistemas monoprocesadores como en sistemas multiprocesadores.

- Los sistemas operativos realizan planificación de procesos a corto, medio y largo plazo. La planificación a corto plazo la realiza el núcleo o kernel. Para aprovechar mejor el procesador, cuando un proceso realiza una operación de E/S, se realiza un cambio de contexto para pasar a ejecutar otro proceso. La planificación a medio plazo se encarga del paso de procesos de la memoria principal a la secundaria y viceversa. La planificación a largo plazo se encarga de la preparación de nuevos procesos para que comience su ejecución, y de los procesos que han terminado su ejecución.

RESUMEN

- Los sistemas multiprocesadores pueden ser fuertemente acoplados cuando los procesadores comparten memoria, o débilmente acoplados si no la comparten. Un tipo particular de sistemas débilmente acoplados son los sistemas distribuidos.
- Un hilo pertenece a un proceso. Un proceso tiene, inicialmente, un hilo de ejecución, pero puede crear más sobre la marcha. Los distintos hilos de un mismo proceso comparten la memoria, por lo que la creación de un nuevo hilo es mucho más rápida que la de un nuevo proceso. El planificador a corto plazo o CPU scheduler gestiona de manera independiente los distintos procesos de un mismo hilo.
- Un servicio es un tipo particular de proceso que se ejecuta en segundo plano (en background), no muestra ninguna interfaz de usuario, y con el que, por lo tanto, no interactúan directamente los usuarios, y que proporciona servicios a otros procesos que pueden ser procesos locales (que se ejecutan en el mismo sistema) o remotos (que se ejecutan en otros sistemas y se comunican con los servicios mediante protocolos de red).

RESUMEN

- La clase `Process` permite gestionar procesos. Es una clase abstracta, y sus instancias se pueden crear con métodos de las clases `Runtime` y `ProcessBuilder`.
- La clase `ProcessBuilder` tiene métodos para redirigir la entrada y salida de los procesos que crea.
- La clase `Process` tiene métodos que devuelven streams que permiten enviar datos directamente a la entrada estándar de un proceso, y leer de su salida estándar y de error.

Ejercicios propuestos

- Crea un programa que muestre un prompt `cuentapalfich>` y que, para cada nombre de fichero introducido, muestre el número de líneas, palabras y caracteres obtenidos mediante una llamada al comando `wc`. Para cada nombre de fichero proporcionado, debe dirigirse la entrada estándar de un proceso creado para el comando `wc` desde el fichero indicado. La ejecución del programa terminará cuando se introduzca una línea vacía. La salida estándar y la salida de error del proceso creado deben dirigirse hacia las del proceso padre, es decir, hacia las del programa desarrollado, mediante los métodos `redirectOutput` y `redirectError`.
- Crea un programa que debe funcionar como el anterior desde el punto de vista del usuario, pero que, en lugar de mostrar directamente la salida del comando `wc`, lo muestre de la siguiente forma: `fichero tiene num_lin líneas, num_pal palabras y num_car caracteres`. Para ello, debe obtenerse un `InputStream` asociado a la salida estándar del proceso. Para obtener el número de líneas, palabras y caracteres a partir de la salida del proceso `wc`, puedes utilizar el método `split` de la clase `String`.

Ejercicios propuestos

- • Crea un programa que muestre un prompt `cuentapalenlin>` y permita introducir líneas hasta que se introduzca una línea vacía. Para cada línea introducida, se debe mostrar el número de palabras y caracteres obtenido de un proceso creado para el comando `wc`, a cuya entrada se dirigirá el contenido de la línea leída. La entrada al proceso se debe realizar utilizando un `OutputStream` obtenido mediante `getOutputStream()`. La salida de `wc` se puede mostrar sin cambios, tal como la genera el comando `wc`.
- • Crea un programa que muestre un prompt `cuentypalpc>`. Es similar al anterior, pero deben leerse líneas hasta que se introduzca una línea vacía. El comando `wc` debe proporcionar información para el conjunto de líneas introducidas hasta antes de introducirse la última línea, que estará vacía. Se debe crear un solo proceso para el comando `wc`, y se le deben dirigir todas las líneas leídas, una a una, hasta que se introduzca una línea vacía. Entonces, debe mostrarse la información correspondiente al conjunto de líneas que se han introducido. La entrada al proceso se debe realizar también utilizando un `OutputStream` obtenido mediante `getOutputStream()`.